Name: Shyam Mohan Verma

Roll No: M25MAC010

PROBLEM 1: Learning Word Embeddings from IIT Jodhpur Data

1. Objective

The objective of this assignment is to train and compare **Word2Vec models (CBOW and Skip-gram)** on IIT Jodhpur textual data and evaluate how effectively they capture **semantic relationships** between words.

2. Dataset Preparation

2.1 Data Sources

The dataset was constructed from:

- IIT Jodhpur official website
- Department webpages
- Faculty-related content
- Academic regulation PDF

2.2 Dataset Statistics

- **Total Documents:** 4
- **Total Tokens:** 7108
- **Vocabulary Size:** 1924

2.3 Preprocessing Steps

- Lowercasing
- Tokenization
- Stopword removal
- Punctuation removal
- Cleaning noisy text

2.4 Observations

- Dataset is **small**, limiting embedding quality
- Presence of **domain-specific academic vocabulary**
- Some **noisy tokens** observed (e.g., merged words)

3. Model Implementation

Two Word2Vec variants were implemented:

- **CBOW (Continuous Bag of Words)**
- **Skip-gram**

3.1 Training Configurations

Configuration 1

- Embedding Dimension: 50
- Window Size: 2
- Negative Sampling: 2

Configuration 2

- Embedding Dimension: 100
- Window Size: 3
- Negative Sampling: 5

Configuration

- Embedding Dimension: 300
- Window Size: 3
- Negative Sampling: 5

4. Training Results

4.1 Loss Values

Configuration 1

- CBOW Final Loss: **53631.5412**
- Skip-gram Final Loss: **58490.3184**

Configuration 2

- CBOW Final Loss: **53665.7654**

- Skip-gram Final Loss: **164060.8897**

Configuration 3

- CBOW Final Loss: **53653.2939**
- Skip-gram Final Loss: **162496.8349**

4.2 Analysis of Loss

- CBOW shows **stable performance across configurations**
- Skip-gram loss increases significantly in higher configuration
- Larger embedding size and negative sampling:
    - Do **not improve CBOW significantly**
    - Make Skip-gram **harder to train**

Reason:
Skip-gram requires **larger datasets**, and current data is insufficient.

5. Semantic Analysis

5.1 Custom CBOW Model

Most similar words to "student":

- phd (0.3853)
- fresh (0.3757)
- credits (0.3712)
- performance (0.2982)
- days (0.2974)

Observation:

- Captures partial academic context
- Similarity scores are **low (~0.3)**
- Some irrelevant associations exist

5.2 Gensim CBOW Model

Most similar words to "student":

- degree (0.9604)
- semester (0.9596)

- requirements (0.9565)
- course (0.9532)
- research (0.9522)
- academic (0.9502)
- program (0.9493)

Observation:

- Strong semantic grouping
- Very high similarity scores (~0.95)
- Clearly captures academic relationships

Conclusion:
Gensim model significantly outperforms custom implementation.

6. Nearest Neighbor Analysis

6.1 Word: "research"

- parttime
- detection
- goal

Mixed relevance → noisy embeddings

6.2 Word: "student"

- phd
- credits
- performance

Academic context captured

6.3 Word: "phd"

- mtech
- students
- degree
- dual

Strong academic clustering

6.4 Vocabulary Issue

- "exam" not found
- Used "examination" instead

Indicates:

- Lack of normalization
- Need for lemmatization

7. Visualization Analysis (PCA & t-SNE)

To evaluate embeddings, PCA and t-SNE were used.

7.1 Skip-gram Visualization

t-SNE

- Partial clustering observed:
    - *student, course, degree, phd* → academic cluster
    - *faculty, lab, teaching* → institutional cluster
- Some words isolated (*research*)

Interpretation:
Captures relationships but clusters are not tight

PCA

- Linear spread of words
- Some relationships visible:
    - student–course
    - phd–degree

Interpretation:
Preserves global structure but weak clustering

7.2 CBOW Visualization

t-SNE

- Words are scattered
- Weak grouping:

    - faculty–teaching–lab

Interpretation:
Poor semantic grouping

PCA

- Small clusters:
    - project–research–faculty
    - learning–education

Interpretation:
Captures limited thematic similarity

7.3 PCA vs t-SNE

Aspect	PCA	t-SNE
Type	Linear	Non-linear
Focus	Global structure	Local similarity
Clustering	Weak	Strong

7.4 Final Visualization Insight

- Skip-gram shows **better clustering than CBOW**
- Overall clusters are **weak due to small dataset**

8. Comparison: CBOW vs Skip-gram

Feature	CBOW	Skip-gram
Training Speed	Faster	Slower
Performance (small data)	Better	Poor
Semantic detail	Moderate	Better (in theory)
Stability	High	Low

Conclusion:
CBOW is more suitable for this dataset.

9. Challenges Faced

- Small dataset size
- Noisy scraped data
- Token merging issues
- Vocabulary inconsistencies
- Missing words (e.g., exam)

10. Conclusion

- Successfully trained Word2Vec models on IIT Jodhpur data
- CBOW performed better due to small dataset
- Skip-gram struggled with high loss
- Gensim model produced significantly better embeddings
- Visualization confirms limited but meaningful clustering

11. Future Work

- Increase dataset size
- Apply better preprocessing:
    - Lemmatization
    - Cleaning merged tokens
- Use advanced models:
    - FastText
    - BERT

The experiment demonstrates that **embedding quality is highly dependent on dataset size and preprocessing**, and while CBOW performs better on small datasets, **Skip-gram requires larger corpora to effectively capture semantic relationships**.

12. Tools & Libraries Used

- Python
- NLTK
- BeautifulSoup

- Requests
- PDFPlumber
- NumPy
- Matplotlib
- Gensim

PROBLEM 2: Character-Level Name Generation using RNN Variants

1. Objective

The objective of this assignment is to design and compare different **recurrent neural network architectures** for generating Indian names at the **character level**, and evaluate their performance using quantitative and qualitative metrics.

2. Dataset

2.1 Dataset Description

- A dataset of **1000 Indian names** was generated using LLMs and stored in TrainingNames.txt.

2.2 Preprocessing Steps

- Converted all names to lowercase
- Added special token "." to mark start and end of names
- Created **character vocabulary**
- Built mappings:
    - Character → Index (stoi)
    - Index → Character (itos)
- Converted names into **input-output character sequences**

3. Model Implementation

Three models were implemented from scratch:

3.1 Vanilla RNN

Architecture

- Embedding layer → RNN → Fully Connected layer
- Predicts next character based on previous sequence

Parameters

- Total Parameters: **37,393**
- Model Size: **0.1426 MB**

3.2 Bidirectional LSTM (BLSTM)

Architecture

- Embedding → Bidirectional LSTM → Fully Connected
- Processes sequence in both forward and backward directions

Parameters

- Total Parameters: **270,737**
- Model Size: **1.0328 MB**

3.3 RNN with Attention

Architecture

- Embedding → RNN → Attention layer → Fully Connected
- Attention assigns weights to important characters

Parameters

- Total Parameters: **37,522**
- Model Size: **0.1431 MB**

3.4 Hyperparameters

- Hidden Size: **128**
- Learning Rate: **0.001**
- Epochs: **20**
- Sampling:
    - Temperature = 0.8
    - Top-k = 5

4. Training Results

4.1 Vanilla RNN

- Final Loss ≈ **0.5144**

Stable convergence, but not very low loss

4.2 BLSTM

- Final Loss ≈ **2.43×10^{-8}**

Extremely low loss → **overfitting likely**

4.3 Attention RNN

- Final Loss ≈ **0.2845**

Best balance between learning and generalization

5. Quantitative Evaluation

5.1 Metrics

- **Novelty Rate**
  → Percentage of generated names not in training set
- **Diversity**
  → Unique names / total generated names

5.2 Results

Model	Novelty Rate	Diversity
Vanilla RNN	1.0000	0.3050
BLSTM	1.0000	0.2100
Attention RNN	1.0000	0.2150

5.3 Analysis

- All models achieved 100% novelty
  - Generated names are not direct copies
- Vanilla RNN has highest diversity (0.305)
  - Produces more varied names
- BLSTM has lowest diversity
  - Indicates overfitting + repetition
- Attention model improves over BLSTM but still less diverse than RNN

6. Qualitative Analysis

6.1 Sample Generated Names

- Aaravv
- Aadhyi
- Aadhyd
- Aadhyr
- Aaras
- Aarrv
- Aadhyan
- Aaravn
- Aaradhyr
- Aaran

6.2 Observations

- Names resemble Indian phonetic patterns
- Common prefixes:
  - "Aa", "Aar", "Aadh"
- Generated names look realistic but slightly distorted

7. Model-wise Behavior

7.1 Vanilla RNN

- Produces diverse names
- Less repetitive
- Slightly noisy outputs

7.2 BLSTM

- Produces highly repetitive outputs:
  - "Viv", "Di.adi" repeated
- Memorizes training data
- Poor generalization

7.3 Attention RNN

- Generates structured names
- Better than BLSTM
- Still some repetition

8. Common Failure Modes

8.1 Repetition Errors

Examples:

- Aanavya
- Aanadhya
- Aaaditya

Cause:

- Model overemphasizes frequent characters

8.2 Duplicate Names

Examples:

- Aaravv
- Aadhyi
- Aarrv

Cause:

- Low diversity in sampling

8.3 Character Duplication

- "Aaa", "rr", "vv" patterns

8.4 Invalid Patterns

- "Di.shaurya" (invalid punctuation)

9. Final Comparison

Model	Strength	Weakness
Vanilla RNN	High diversity	Less accuracy
BLSTM	Very low loss	Overfitting
Attention RNN	Balanced	Moderate diversity

10. Conclusion

- Vanilla RNN performs best in terms of **diversity**
- BLSTM suffers from **overfitting**, despite very low loss
- Attention RNN provides **best trade-off** between structure and variation

While BLSTM achieves extremely low training loss, it fails to generalize and produces repetitive outputs. In contrast, the Vanilla RNN achieves higher diversity, and the Attention-based RNN provides a better balance between realism and generalization.